

Image Denoising using Convolutional Autoencoders
Deep Learning Project Report

Submitted by:

102319008 Kshitiz Chaudhary

102319083 Dwarkesh Rathore

BE Third Year- EEC

Under the Mentorship of
Dr. Mukesh Dalal



Electrical and Computer Engineering
Thapar Institute of Engineering and Technology, Patiala

Session: Jan-May, 2026

Abstract

This report presents a full-stack deep learning web application developed for the ULC665 course at Thapar Institute of Engineering and Technology. The system implements two distinct applications of Convolutional Autoencoders: (1) Image Denoising, which removes Gaussian noise from corrupted images using a trained Convolutional Denoising Autoencoder (CDAE), and (2) Anomaly Detection, which identifies images that deviate significantly from a learned normal class by analysing reconstruction error against a computed threshold.

The denoising model is trained on the CIFAR-10 dataset using Mean Squared Error (MSE) loss with an adaptive learning rate schedule. The anomaly detection model is trained exclusively on automobile images, enabling it to flag non-automobile images as anomalies based on high reconstruction error. Both models are served through a Flask REST API and accessed via an interactive browser-based frontend featuring a dual-tab interface, before/after image comparison, a colour-coded error heatmap, and quantitative evaluation metrics. The project directly covers topics from the ULC665 syllabus including Autoencoders, Convolutional Neural Networks, Batch Normalization, Backpropagation, and Regularization.

1. Introduction

Digital images are a fundamental medium for communication, scientific analysis, and decision-making across virtually every domain. However, images are frequently degraded by noise introduced during acquisition, transmission, or compression. Noise manifests as random pixel distortions that reduce visual quality and adversely affect downstream tasks such as object detection, medical diagnosis, and quality inspection.

Simultaneously, the ability to detect anomalies — images or patterns that deviate from an expected norm — is of critical importance in industrial inspection, medical imaging, fraud detection, and security surveillance. Traditional rule-based methods for both denoising and anomaly detection are brittle, domain-specific, and fail to generalise. Deep learning, specifically the Autoencoder architecture, offers a powerful and generalisable solution to both problems.

An Autoencoder is an unsupervised neural network that learns to compress input data into a compact latent representation and reconstruct it back to the original. A Denoising Autoencoder is trained to reconstruct clean images from noisy inputs, forcing it to learn robust, noise-invariant features. For anomaly detection, an Autoencoder trained only on normal data will reconstruct normal inputs accurately but fail on anomalous inputs — producing a high reconstruction error that serves as an anomaly score.

This project implements both applications as a unified web system. The Flask backend serves two trained Convolutional Autoencoder models, while the HTML/CSS/JavaScript frontend provides an intuitive dual-tab interface for real-time interaction. The system was built using TensorFlow, Keras, NumPy, and Pillow, and trained on the CIFAR-10 dataset.

2. Objectives

The objectives of this project are:

- To design, train, and deploy a Convolutional Denoising Autoencoder that removes Gaussian noise from images, using MSE loss and an adaptive learning rate schedule, served through a Flask REST API.
- To implement an Autoencoder-based Anomaly Detection system trained exclusively on a single normal class (automobile), capable of flagging unseen images as normal or anomalous based on reconstruction error threshold.
- To develop a full-stack web application with a dual-tab interactive frontend that displays quantitative evaluation metrics including reconstruction error, anomaly threshold, confidence score, and a colour-coded error heatmap.
- To demonstrate practical application of ULC665 syllabus concepts — specifically Autoencoders, CNNs, Batch Normalization, Backpropagation, and Regularization — through a deployable, real-world system.

3. Deep Learning Concepts Used

3.1 Autoencoders

An Autoencoder is an unsupervised neural network trained to reconstruct its own input. It consists of two components: an Encoder that compresses the input into a lower-dimensional latent vector, and a Decoder that reconstructs the original input from the latent vector. The network is trained to minimise the reconstruction loss between input x and output $\hat{x}$.

$$(1) \quad z = f_{\text{enc}}(x) \quad [\text{Encoder: input} \rightarrow \text{latent vector}]$$

$$(2) \quad \hat{x} = f_{\text{dec}}(z) \quad [\text{Decoder: latent vector} \rightarrow \text{reconstruction}]$$

$$(3) \quad L = (1/N) * \sum \|x_i - \hat{x}_i\|^2 \quad [\text{MSE Reconstruction Loss}]$$

The compressed latent space forces the model to retain only the most essential structural information about the input, discarding noise and irrelevant variation. This property is exploited in both denoising and anomaly detection.

3.2 Denoising Autoencoder (DAE)

A Denoising Autoencoder is a variant where the model receives a corrupted version of the input but is trained to reconstruct the original clean version. Gaussian noise is added during training to create input-target pairs (noisy, clean). The model learns to recover clean signals from corrupted observations.

Gaussian noise is added as follows:

$$(4) \quad \tilde{x} = \text{clip}(x + \eta * N(0,1), 0, 1)$$

Where x is the clean image, η is the noise factor (0.25 in this project), $N(0,1)$ is standard Gaussian noise, and clip constrains values to $[0,1]$. The model is trained with input $\tilde{x}$ and target x .

3.3 Convolutional Neural Network (CNN) Layers

This project uses Conv2D layers throughout both the encoder and decoder. The 2D convolution operation slides a learnable $k \times k$ filter across the input feature map, computing element-wise products and summing them at each spatial position:

$$(5) \quad (I * K)(i,j) = \text{SUM}_m \text{SUM}_n I(i+m, j+n) * K(m,n)$$

Where I is the input feature map and K is the learnable kernel. All Conv2D layers in this project use 3×3 kernels with 'same' padding (output size = input size) and ReLU activation. The ReLU activation function is:

$$(6) \quad \text{ReLU}(x) = \max(0, x)$$

MaxPooling2D with a 2×2 window halves spatial dimensions by selecting the maximum value in each block — retaining strong features and suppressing weak noise. UpSampling2D reverses this in the decoder by repeating each pixel across a 2×2 grid. The encoder uses increasing filter counts ($32 \rightarrow 64 \rightarrow 128$) to capture increasingly abstract features, while the decoder mirrors this in reverse ($128 \rightarrow 64 \rightarrow 32 \rightarrow 1$).

3.4 Batch Normalization

Batch Normalization (BN) normalises the output of a layer across the current mini-batch to have zero mean and unit variance, reducing Internal Covariate Shift. Given a mini-batch of activations $B = \{x_1, \dots, x_m\}$, BN computes:

$$(7) \quad \mu_B = (1/m) * \text{SUM } x_i \quad \text{[Mini-batch mean]}$$

$$(8) \quad \sigma^2_B = (1/m) * \text{SUM } (x_i - \mu_B)^2 \quad \text{[Mini-batch variance]}$$

$$(9) \quad \hat{x}_i = (x_i - \mu_B) / \sqrt{\sigma^2_B + \epsilon} \quad \text{[Normalised output]}$$

$$(10) \quad y_i = \gamma * \hat{x}_i + \beta \quad \text{[Scale and shift]}$$

Where ϵ is a small constant ($\sim 1e-5$) for numerical stability, and γ and β are learnable parameters. BN is applied after each Conv2D layer in the encoder. It stabilises training, enables higher learning rates, and acts as a mild regulariser by introducing noise through batch-level statistics.

3.5 Anomaly Detection Using Autoencoders

Autoencoder-based anomaly detection exploits a key property: a model trained exclusively on normal data will reconstruct normal inputs accurately but fail on anomalous ones, since the latter fall outside the learned distribution. The reconstruction error serves as an anomaly score.

For each input image x , the reconstruction error (per-image MSE) is computed as:

$$(11) \quad E(x) = (1/HWC) * \sum_{\{h,w,c\}} (x_{\{h,w,c\}} - \hat{x}_{\{h,w,c\}})^2$$

Where H , W , C are the height, width, and channels of the image. The anomaly threshold T is set using the training set reconstruction errors:

$$(12) \quad T = \mu_{\text{train}} + 2 * \sigma_{\text{train}}$$

Where μ_{train} and σ_{train} are the mean and standard deviation of reconstruction errors across all normal training images. An image is classified as:

$$(13) \quad \text{Verdict} = \text{ANOMALY} \text{ if } E(x) > T, \text{ else NORMAL}$$

In this project, the anomaly model is trained only on automobile images (class 1 of CIFAR-10). Any image whose reconstruction error exceeds the threshold is flagged as an anomaly. A per-pixel error map is also computed and visualised as a red-green heatmap, where red regions indicate areas of high reconstruction error.

4. System Architecture

4.1 Denoising Autoencoder Model Architecture

Both the denoising and anomaly detection models share the same Convolutional Autoencoder architecture, operating on 64 x 64 greyscale images (shape: 64 x 64 x 1).

Layer	Type	Filters	Output Shape
Input	Input	—	64 x 64 x 1
Enc-Conv1	Conv2D + BN	32	64 x 64 x 32
Enc-Pool1	MaxPooling2D	—	32 x 32 x 32
Enc-Conv2	Conv2D + BN	64	32 x 32 x 64
Enc-Pool2	MaxPooling2D	—	16 x 16 x 64
Enc-Conv3	Conv2D + BN	128	16 x 16 x 128
Enc-Pool3	MaxPooling2D	—	8 x 8 x 128
Bottleneck	Conv2D	128	8 x 8 x 128
Dec-Up1	UpSampling2D	—	16 x 16 x 128
Dec-Conv1	Conv2D	64	16 x 16 x 64
Dec-Up2	UpSampling2D	—	32 x 32 x 64
Dec-Conv2	Conv2D	32	32 x 32 x 32
Dec-Up3	UpSampling2D	—	64 x 64 x 32
Output	Conv2D (sigmoid)	1	64 x 64 x 1

4.2 Application Architecture

The project follows a standard client-server architecture with three layers:

- Frontend (HTML/CSS/JavaScript): Dual-tab interface providing Image Denoising and Anomaly Detection tabs. Handles image upload, noise slider, result display, error heatmap, confidence bar, and metric cards.
- Backend (Flask/Python): Exposes two REST endpoints — /denoise and /anomaly — each accepting POST requests with image files. Loads trained models at startup and returns base64-encoded result images with metric data as JSON responses.
- Models (Keras/TensorFlow): Two independently trained autoencoders. The denoising model is trained on full CIFAR-10 with noise addition. The anomaly model is trained only on automobile images with identity mapping (input = target, no noise).

5. Technology Stack

Component	Technology	Purpose
Deep Learning	Keras + TensorFlow 2.x	Model building, training, inference
Backend Server	Flask 3.0	REST API, image preprocessing, model serving
Frontend UI	HTML5 / CSS3 / JavaScript	Dual-tab interface, upload, visualisation
Dataset	CIFAR-10 (Keras built-in)	Training data — denoising (all) + anomaly (automobiles only)
Image Processing	Pillow (PIL)	Greyscale conversion, resizing, base64 encoding
Numerics	NumPy	Noise generation, array ops, error map computation
Optimiser	Adam + ReduceLROnPlateau	Adaptive weight updates with decaying learning rate
Loss Function	MSE (Denoising), MSE (Anomaly)	Pixel-level reconstruction error minimisation
IDE	VS Code + PowerShell	Development, virtual environment, execution

6. Implementation Details

6.1 Dataset Preparation

The CIFAR-10 dataset contains 60,000 colour images (32 x 32 x 3) across 10 classes. All images are converted to greyscale and resized to 64 x 64 using Pillow's LANCZOS resampling, then normalised to [0, 1]:

$$(14) \quad \mathbf{x_norm} = \mathbf{x_raw} / 255.0$$

For the denoising model, all 50,000 training images are used. For the anomaly detection model, only the 5,000 automobile images (class label = 1) from the training set are retained. This ensures the anomaly model only learns the distribution of a single normal class.

6.2 Denoising Model Training

The denoising model is compiled with the Adam optimiser and MSE loss function — a change from Binary Cross-Entropy which was used initially. MSE is more appropriate for image reconstruction as it directly measures pixel-level differences:

$$(15) \quad L_MSE = (1/N) * \sum_{i=1}^N (\mathbf{x_i} - \hat{\mathbf{x_i}})^2$$

A ReduceLROnPlateau callback monitors validation loss and halves the learning rate when no improvement is seen for 3 consecutive epochs. This prevents the model from overshooting optima at later training stages:

$$(16) \quad \mathbf{lr_new} = \mathbf{lr_current} * 0.5 \quad \text{if val_loss stagnates for 3 epochs}$$

Training hyperparameters: 25 epochs, batch size 32, noise factor $\eta = 0.25$. Trained weights are saved to weights/autoencoder_cifar.h5 after training, enabling instant loading on subsequent server restarts.

6.3 Anomaly Detection Model Training

The anomaly model is trained using identity mapping — the input and target are identical clean automobile images with no noise added. The model learns to reconstruct automobiles accurately. MSE loss is used:

$$(17) \quad L_{\text{anomaly}} = (1/N) * \text{SUM} (x_{\text{auto}_i} - \hat{x}_{\text{auto}_i})^2$$

After training for 20 epochs with batch size 32, the threshold T is computed from the training set reconstruction errors using equation (12). This threshold is saved to `weights/anomaly_threshold.json` alongside the model weights. At inference, the per-pixel error map is computed and normalised to $[0, 1]$ for heatmap visualisation, where red pixels indicate high reconstruction error and green pixels indicate low error.

6.4 Flask REST Endpoints

The Flask application exposes two POST endpoints:

/denoise

Accepts an optional image file and `noise_level` parameter (default 0.25). Preprocesses the image, adds Gaussian noise, runs the denoising model, and returns three base64-encoded PNG images: original, noisy, and denoised.

/anomaly

Accepts an optional image file. Preprocesses and runs the image through the anomaly model. Returns: original image, reconstructed image, error heatmap, reconstruction error score, threshold value, NORMAL/ANOMALY verdict, and confidence percentage. If no file is uploaded, a random CIFAR-10 image is used for demonstration.

6.5 Frontend Interface

The frontend provides two tabs — Image Denoiser and Anomaly Detection — accessible via a tab bar at the top of the page. Key frontend features include: drag-and-drop image upload with visual feedback, noise level slider with real-time value display (denoising tab), three-panel image comparison grid (original, processed, result),

confidence bar with colour coding (green for NORMAL, red for ANOMALY), metric cards displaying reconstruction error, threshold, and confidence score, and a status bar showing processing result after each request.

7. Results and Observations

7.1 Denoising Results

After switching to MSE loss and adding the ReduceLROnPlateau callback, the model achieved significantly lower loss values — typically in the range of 0.003 to 0.010 per epoch compared to 0.58 with Binary Cross-Entropy. Key observations:

- The model successfully reconstructed digit and object shapes from heavily noisy inputs at noise levels up to 0.7.
- The ReduceLROnPlateau callback triggered learning rate reductions in later epochs, visibly improving convergence stability.
- Uploaded real-world images (photos, screenshots) were denoised effectively, with the model generalising beyond its CIFAR-10 training distribution.
- Response time per inference was approximately 200-400ms, well within acceptable limits for a real-time web application.

7.2 Anomaly Detection Results

The anomaly detection model demonstrated clear separation between normal and anomalous inputs:

- Automobile images produced low reconstruction errors (below threshold) → correctly classified as NORMAL.
- Non-automobile images (cats, birds, planes, etc.) produced high reconstruction errors (above threshold) → correctly classified as ANOMALY.
- The error heatmap visually confirmed which regions of anomalous images the model struggled to reconstruct — typically the distinctive shape and texture regions that differ from cars.
- The true class label (from CIFAR-10) was displayed in the status bar for random demo images, allowing direct verification of the anomaly verdict.

8. Alignment with ULC665 Syllabus

Syllabus Topic	Application in This Project
Autoencoders	Core architecture for both denoising and anomaly detection models
Denoising Autoencoder	Directly implemented — model trained on noisy/clean image pairs
Convolutional Neural Networks	Conv2D layers used throughout encoder and decoder stages
Batch Normalization	Applied after all encoder Conv2D layers for stable training
Regularization Methods	BN as regulariser; ReduceLROnPlateau prevents overfitting at late epochs
Backpropagation	Used during training via Adam optimiser with MSE loss
Architecture Design	Encoder → Bottleneck → Decoder — standard deep network design
Generative / Reconstruction	Decoder reconstructs clean images from compressed latent space
Unsupervised Learning	Both models trained without class labels — only image data used

9. Conclusion

This project successfully demonstrates two practical applications of Convolutional Autoencoders — Image Denoising and Anomaly Detection — delivered as a unified, fully functional web application. The system integrates key concepts from the ULC665 Deep Learning curriculum and extends them into a deployable, interactive system accessible from any browser.

The transition from Binary Cross-Entropy to MSE loss, combined with the ReduceLROnPlateau adaptive learning rate schedule, significantly improved denoising performance. The anomaly detection module adds a genuinely distinct application of the autoencoder architecture, exploiting reconstruction error as an unsupervised anomaly signal — a technique with direct real-world relevance in industrial inspection, medical imaging, and security surveillance.

The dual-tab Flask web application demonstrates how deep learning models can be taken from training scripts to production-ready deployable systems — a skill of direct relevance to modern AI engineering roles. Future enhancements could include colour image support, U-Net skip connections for improved reconstruction quality, support for multiple normal classes in anomaly detection, and cloud deployment on platforms such as AWS or Heroku.

10. References

- [1] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. MIT Press, 2016.
- [2] F. Chollet, Deep Learning with Python. Manning Publications, 2018.
- [3] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, and P. A. Manzagol, "Stacked Denoising Autoencoders," Journal of Machine Learning Research, vol. 11, 2010.
- [4] D. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," ICLR, 2015.
- [5] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training," ICML, 2015.

- [6] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," Technical Report, University of Toronto, 2009. (CIFAR-10 Dataset)
- [7] TensorFlow Documentation. https://www.tensorflow.org/api_docs
- [8] Flask Documentation. <https://flask.palletsprojects.com>
- [9] Keras API Reference. <https://keras.io/api/>